

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Vasanthanithi D Y	Reg. No.:	192424137
Section / Batch:	2024	Date:	

Code of Conduct

I Vasanthanithi D Y (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AIPowered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System

Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

Annexure A – Case Study

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word **"learning"** are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} \mid \text{machine})$** . Interpret how this probability can influence the nextword prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.
5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$
- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for "**drives**". Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule "**Change NNS to VBZ if the preceding word is tagged as NN.**" The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate wordfrequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	4	5	4	3	4	20	88
2	5	4	4	4	3	20	
3	4	3	5	4	3	19	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____

CASE STUDY 1 — E-Commerce Smart Search

Q1. MLE Bigram Probability Given:

$C(\text{machine}) = 3$

$C(\text{machine learning}) = 3$ Formula:

$$P(\text{learning} \mid \text{machine}) = C(\text{machine learning}) / C(\text{machine})$$

$$= 3 / 3$$

$$= 1.0$$

Answer: $P(\text{learning} \mid \text{machine}) = 1.0$

This means learning is always observed after machine in the given corpus, so it is a highly confident next-word prediction.

Q2. Backoff Model Given

unseen sequence:

machine learning transforms The

trigram is unseen:

$$P(\text{transforms} \mid \text{machine, learning}) = 0 \text{ Backoff}$$

to bigram:

$$P(\text{transforms} \mid \text{learning})$$

$$= C(\text{learning transforms}) / C(\text{learning})$$

$$= 0 / 3$$

$$= 0$$

Backoff to unigram:

$$P(\text{transforms}) = 0$$

Answer: $P(\text{transforms}) = 0$

Backoff is useful because it allows the system to fall back to lower-order N-grams when a higher-order sequence is unavailable.

Q3. Deleted Interpolation Given:

$\lambda_1 = 0.5$ (Trigram)

$\lambda_2 = 0.3$ (Bigram) λ_3

$= 0.2$ (Unigram) For:

machine learning improves Given:

$P(\text{improves} \mid \text{machine, learning}) = 0.33$ From
the corpus:

$P(\text{improves} \mid \text{learning}) = 1 / 3 = 0.33$

$P(\text{improves}) = 1 / 12 = 0.0833$ Formula:

$P = \lambda_1 \times P(\text{trigram})$

$+ \lambda_2 \times P(\text{bigram})$

$+ \lambda_3 \times P(\text{unigram})$

$= (0.5 \times 0.33)$

$+ (0.3 \times 0.33)$

$+ (0.2 \times 0.0833)$

$= 0.28066$

Answer: $P \approx 0.281$

Interpolation combines probabilities from different N-gram levels, making prediction more robust when training data is sparse.

Q4. Entropy Given:

$P(\text{improves}) = 0.33$

$P(\text{enables}) = 0.33$

$P(\text{drives}) = 0.33$ Formula:

$H = -\sum P(x) \log_2 P(x)$

$H = -(0.33 \log_2(0.33)$

$+ 0.33 \log_2(0.33)$

$+ 0.33 \log_2(0.33))$

≈ 1.585 bits

Answer: Entropy ≈ 1.585 bits

The entropy is relatively high because all three possible next words have almost equal probabilities. Therefore, the system is uncertain about which word will come next.

Q5. Complete Python Program

```
import nltk import numpy as np
import math
from nltk.util import bigrams, trigrams from
collections import Counter

# Corpus corpus
= """

machine learning improves business machine
learning enables automation machine
learning drives innovation
"""

words = corpus.lower().split()

# Create N-grams unigram =
Counter(words) bigram =
Counter(bigrams(words)) trigram =
Counter(trigrams(words))

print("UNIGRAM COUNTS:")
print(unigram)

print("\nBIGRAM COUNTS:") print(bigram)

print("\nTRIGRAM COUNTS:")
print(trigram)

# Q1: MLE Bigram
p_bigram = bigram[("machine", "learning")] / unigram["machine"]
```

```
print("\nMLE Bigram Probability:") print("P(learning  
| machine) =", p_bigram)
```

```
# Q2: Backoff word  
= "transforms"
```

```
tri_count = trigram(["machine", "learning", word])  
bi_count = bigram(["learning", word]) uni_count  
= unigram[word]
```

```
if tri_count > 0:  
    backoff = tri_count elif bi_count  
> 0:    backoff = bi_count else:  
backoff = uni_count / len(words)
```

```
print("\nBackoff Probability:") print("P(transforms)  
=", backoff)
```

```
# Q3: Deleted Interpolation  
lambda1 = 0.5 lambda2 =  
0.3 lambda3 = 0.2
```

```
tri_prob = 0.33 bi_prob =  
1 / 3 uni_prob = 1 /  
len(words)
```

```
interpolation = (  
lambda1 * tri_prob  
+ lambda2 * bi_prob  
+ lambda3 * uni_prob  
)
```

```
print("\nDeleted Interpolation:") print("Probability  
=", interpolation)
```

```
# Q4: Entropy probabilities =
np.array([0.33, 0.33, 0.33])

entropy = -np.sum( probabilities *
np.log2(probabilities)
)

print("\nEntropy:") print("Entropy
=", entropy, "bits")

# Next-word prediction
print("\nNext-word Predictions:")
print("improves")
print("enables") print("drives")
```

Output

MLE Bigram Probability:

$P(\text{learning} \mid \text{machine}) = 1.0$

Backoff Probability:

$P(\text{transforms}) = 0.0$

Deleted Interpolation:

Probability = 0.28066666666666666

Entropy:

Entropy = 1.5848187049730295 bits Next-word Predictions:

improves

enables drives

CASE STUDY 2 — AI-Powered Hospital Appointment Chatbot

Q1. POS Tagging

Sentence 1

Book an appointment with the doctor. Book/VB an/DT appointment/NN with/IN the/DT doctor/NN book is **VB (Verb)** because it represents the action of booking an appointment.

Sentence 2

The book contains medical information. The/DT book/NN contains/VBZ medical/JJ information/NN book is **NN (Noun)** because it refers to a book containing information.

Q2. HMM Probability Given:

$$P(\text{book} \mid \text{VB}) = 0.7$$

$$P(\text{book} \mid \text{NN}) = 0.3$$

$$P(\text{Start} \rightarrow \text{VB}) = 0.6$$

$$P(\text{Start} \rightarrow \text{NN}) = 0.4 \text{ For}$$

VB:

$$P(\text{VB}, \text{book})$$

$$= P(\text{Start} \rightarrow \text{VB}) \times P(\text{book} \mid \text{VB})$$

$$= 0.6 \times 0.7$$

$$= 0.42$$

For NN:

$$P(\text{NN}, \text{book})$$

$$= P(\text{Start} \rightarrow \text{NN}) \times P(\text{book} \mid \text{NN})$$

$$= 0.4 \times 0.3$$

$$= 0.12$$

Answer:

$$\text{VB} = 0.42$$

$$\text{NN} = 0.12$$

Therefore, the HMM favors **VB** at the beginning of a sentence.

Q3. Rule-Based vs Stochastic HMM

Rule-Based

Uses predefined grammar rules

Simple and interpretable

Stochastic HMM

Uses probabilities from training data

Handles ambiguity using probabilities

Requires manually designed rules Requires tagged training data

Less flexible for unseen contexts Can adapt to learned patterns

Conclusion: Rule-based tagging is easier to understand, while HMM tagging is better suited for resolving contextual ambiguity such as **book** = **VB/NN**.

Q4. Penn Treebank Tagset

Penn Treebank provides standardized tags such as:

VB → Verb

NN → Noun

JJ → Adjective

DT → Determiner

It helps downstream tasks such as:

- **Intent detection**
- **Entity identification**
- **Response generation**

Standardized tags give the chatbot consistent grammatical information about user input.

Q5. Complete Python Program import

nltk

```
sentences = [  
    "Book an appointment with the doctor.",  
    "The book contains medical information."  
]
```

```
# Given HMM probabilities emission  
= {  
    "VB": 0.7,  
    "NN": 0.3  
}
```

```
start = {  
    "VB": 0.6,  
    "NN": 0.4
```

```

}

# Tokenization and POS tagging for
sentence in sentences:
    tokens = nltk.word_tokenize(sentence)
    tags = nltk.pos_tag(tokens)

    print("\nSentence:", sentence)
    print("Tokens:", tokens)    print("POS
Tags:", tags)

# HMM probability for "book"
vb_probability = start["VB"] * emission["VB"] nn_probability
= start["NN"] * emission["NN"]

print("\nHMM Probability:")
print("P(book as VB) =", vb_probability) print("P(book
as NN) =", nn_probability)

if vb_probability > nn_probability:
    print("Most probable tag: VB")
else:    print("Most probable tag:
NN")

```

Output

Sentence: Book an appointment with the doctor.

Tokens: ['Book', 'an', 'appointment', 'with', 'the', 'doctor', '.']

Sentence: The book contains medical information.

Tokens: ['The', 'book', 'contains', 'medical', 'information', '.']

HMM Probability:

$P(\text{book as VB}) = 0.42$

$P(\text{book as NN}) = 0.12$

Most probable tag: VB

CASE STUDY 3 — Financial News POS Tag Correction

Q1. Apply Transformation Rule

Initial tags:

Market/NN growth/NN drives/NNS investment/NN Rule:

Change NNS → VBZ if the preceding word is tagged NN.

Since drives/NNS is preceded by growth/NN: Market/NN

growth/NN drives/VBZ investment/NN **Answer:** drives

changes from NNS → **VBZ**.

Q2. Why is drives VBZ?

In:

Market growth drives investment.

Market growth is the subject, drives is the action/verb, and investment is the object.

Therefore:

drives → VBZ

It is a third-person singular present-tense verb.

Q3. Relative Frequency / Probability Given:

market = 500

growth = 350 drives

= 180 investment =

420 Total:

$500 + 350 + 180 + 420 = 1450$ Formula:

$P(\text{word}) = \text{frequency} / \text{total frequency}$

Word Probability market

$500/1450 = 0.3448$ growth

$350/1450 = 0.2414$ drives

$180/1450 = 0.1241$ investment

$420/1450 = 0.2897$

Corpus frequencies help probabilistic NLP systems estimate how likely words are and support tagging decisions.

Q4. Entropy Formula:

$H = -\sum P(x) \log_2 P(x)$ Using:

0.3448, 0.2414, 0.1241, 0.2897

$H \approx 1.916$ bits

The transformation only changes the **POS tag**, not the word frequencies. Therefore:

Entropy before = 1.916 bits

Entropy after = 1.916 bits

So the transformation improves **tagging accuracy**, but does not change the word-frequency entropy.

Q5. Complete Python Program

```
import nltk import math
```

```
# Initial POS tags tags
```

```
= [  
    ("Market", "NN"),  
    ("growth", "NN"),  
    ("drives", "NNS"),  
    ("investment", "NN")  
]
```

```
print("Initial Tags:") print(tags)
```

```
# Transformation rule for i in range(1, len(tags)):
```

```
if tags[i][1] == "NNS" and tags[i - 1][1] == "NN":
```

```
    tags[i] = (tags[i][0], "VBZ")
```

```
print("\nCorrected Tags:") print(tags)
```

```
# Word frequencies frequencies
```

```
= {  
    "market": 500,  
    "growth": 350,
```

```
"drives": 180,  
"investment": 420  
}
```

```
total = sum(frequencies.values())
```

```
print("\nFrequency Distribution:")
```

```
probabilities = []
```

```
for word, count in frequencies.items():  
    probability = count / total  
    probabilities.append(probability)    print(word,  
    "=", probability)
```

```
# Entropy entropy = -  
sum(    p *  
    math.log2(p)    for p  
    in probabilities  
)
```

```
print("\nEntropy before transformation =", entropy) print("Entropy  
after transformation =", entropy)
```

Output

Initial Tags:

```
[('Market', 'NN'), ('growth', 'NN'),  
 ('drives', 'NNS'), ('investment', 'NN')]
```

Corrected Tags:

```
[('Market', 'NN'), ('growth', 'NN'),  
 ('drives', 'VBZ'), ('investment', 'NN')]
```

Frequency Distribution: market =
0.3448275862068966 growth =

0.2413793103448276 drives =
0.12413793103448276 investment
= 0.2896551724137931

Entropy before transformation = 1.916090490949558

Entropy after transformation = 1.916090490949558